

GPT Researcher

Deep Research / Multi-Agent / MCP / Context Engineering

完整源码与工程研究报告

研究日期：2026-08-13

研究对象：assafelovic/gpt-researcher v3.6.0 / main

研究方法

以官方 GitHub 主仓库、官方 docs.gptr.dev、release notes 与项目内 .claude/reference 文档为主要一手资料；辅以 Plan-and-Solve、RAG、STORM 与 DeepResearch Bench 等原始论文/基准。报告明确区分“官方事实”“源码推断”“工程评价”，并将项目方自测 benchmark 标注为自报结果。

GPT Researcher 架构演化（研究视角）



注：时间点基于项目 README、官方文档、release 与仓库目录；“演化”表示工程重心变化，不代表旧路径被完全删除。

图 1 GPT Researcher 的工程重心演化（研究性归纳）

执行摘要

GPT Researcher 是 2023 年出现的开源 Research Agent 项目之一。它最值得学习的地方并不是“接了多少搜索 API”，而是较早把一个复杂研究任务分解成：任务角色选择、研究问题生成、检索、全文抓取、相关性筛选、上下文压缩、证据聚合和长报告生成。到 2026 年，这套工程又叠加了递归 Deep Research、MCP 数据源、多 Agent 审核/修订，以及把 GPT Researcher 作为 research engine 嵌入 Deep Agents harness 的路径。

一句话结论

GPT Researcher 的核心资产是“把搜索结果变成可供 LLM 使用的研究上下文”的中间层，而不是搜索本身。真正的 know-how 位于 Query Decomposition、Full-page Scraping、Context Compression、Evidence/Citation 和并行研究编排之间。

- 经典 pipeline：Query → choose_agent → plan_research → 多个 sub-query → Retriever → Scraper → ContextManager → ReportGenerator。
- Deep Research：在经典 pipeline 之上增加 breadth × depth 的递归搜索树、follow-up question、learnings/citations 累积与并发控制。
- Multi-Agent：将“一个报告”拆成章节级并行研究，并引入 Reviewer/Revisor 质量循环；适合高质量长文，但成本更高。
- MCP：把“网页搜索”推广成“任意专业数据源/工具”，通过智能工具选择与 hybrid retrieval 补足领域数据。
- 2026 v3.6.0 的重点是 robustness/security/retriever/scraper hardening，说明项目进入从功能扩张转向工程可靠性的阶段。
- 最适合借鉴的设计：分层研究流水线、Retriever/Scraper 解耦、Context Compression、source tracking、async 并行、工具化 research engine。
- 最需要谨慎的点：来源数量不等于事实正确；多源一致性并非严格事实验证；递归深搜会快速增加成本；citation 数量不能替代 citation precision。

0. 当前项目状态（截至 2026-08-13）

指标	当前状态	意义
Repository	assafelovic/gpt-researcher	主仓库，Apache-2.0
Latest release	v3.6.0（2026-07-18）	当前稳定线索
GitHub Stars	28,953	生态成熟度/关注度信号，不等同于质量
Forks	3,926	二次开发活跃
Primary language	Python	核心 agent/retrieval backend
Core interfaces	PIP package / FastAPI / NextJS / MCP / Multi-Agent	从库到产品多种接入形态
Research sources	Web / local documents / hybrid / MCP	研究数据面较完整

v3.6.0 release 明确强调 security、stability、performance，并包含 retriever hardening、scraper robustness、multi-agent revision loop 约束、成本/配置/MCP 修复，以及与 LangChain Deep Agents 的新示例。这些变化说明当前工程问题已经从“能不能做 Deep Research”转向“异常结果、恶意内容、成本、上下文和循环如何可控”。

1. 为什么 GPT Researcher 在 2023 年有意义：Know-why

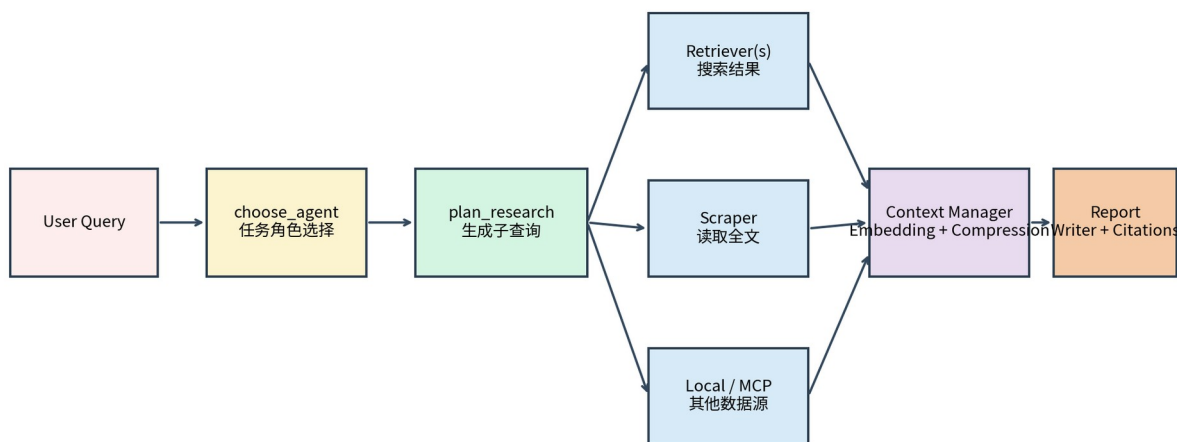
GPT Researcher 官方将其动机归纳为：纯 LLM 的知识会过时、容易 hallucination；研究任务需要超过单次上下文/输出限制的长文本；单一或少量网页来源容易造成浅层结论与偏差；人工研究耗时。这个判断与 RAG 的“外部非参数知识”方向一致，但它进一步把 RAG 从单轮 retrieval 扩展为一个 task-oriented research workflow。

其“Plan-and-Solve”影响也很直接：复杂任务不应该直接从原问题跳到答案，而应先生成一个计划/子问题集，然后分别执行。放到 research 场景里，plan 不再是数学推理步骤，而是多个可检索、可验证、可并行的研究问题。

关键分界

普通 RAG 解决的是“给定问题，找哪些文档”；GPT Researcher 解决的是“为了回答一个复杂研究问题，下一步应该问哪些可检索的问题、读哪些全文、保留哪些证据、何时继续搜索、如何组织成长报告”。

经典 GPTResearcher：从问题到可引用报告的数据流



Sources: :claude/references/flows.md; gpt_researcher/skills/context_manager.py; official docs

图 2 经典 GPTResearcher 数据流（基于当前官方 flow 文档重绘）

2. 核心架构：经典 GPTResearcher Pipeline

2.1 GPTResearcher 是 orchestration root

当前官方 flow 文档显示，GPTResearcher.__init__ 负责加载 config、初始化 ResearchConductor / ReportGenerator 等 skills、按 RETRIEVER 配置构造 retrievers，并在需要时初始化图像生成等附加能力。conduct_research() 负责研究阶段，write_report() 负责报告阶段。这个分离是非常有价值的 API 设计：research context 可以被其他 agent 或系统复用，不必强制生成最终文章。

2.2 choose_agent: 先决定 “以什么角色研究”

在经典模式中，如果没有显式传入 agent/role，系统会让 LLM 根据 query 选择任务相关的 agent role。其价值不是 “角色扮演”，而是给后续 query generation 和 report prompt 提供 domain-specific framing。工程上它相当于一个轻量的 task router / prompt router。

2.3 plan_research: 先展开问题空间

ResearchConductor 会为原问题生成一组 sub-queries。官方 flow 文档描述为通常生成 3-5 个子查询。每个子查询可以独立执行检索和内容筛选，因此天然适合 asyncio 并行。这里的核心不是固定 3 或 5，而是将 “一个不可直接搜索的复杂问题” 变成多个搜索引擎能够有效处理的 query。

Know-how: Query Decomposition 的目标函数

好的 sub-query 不是把原问题换一种说法，而是最大化覆盖差异（coverage diversity）、减少重叠（redundancy）、提高可检索性（retrievability），同时保证所有子问题合起来能够重构原始研究目标。

3. Retriever Layer: 搜索不是单点能力，而是可组合数据面

官方文档显示，GPT Researcher 默认使用 Tavily，但 RETRIEVER 可以切换或组合多个检索器，例如 Tavily、Bing、Google、Arxiv 等；多个 retriever 可用逗号组合。其架构意义在于：Retriever 只负责 “发现候选资源”，而不是承担最终内容理解。

设计	为什么重要	工程收益
Retriever 可插拔	不同领域的最佳搜索源不同	避免绑定单一搜索供应商
多 Retriever 组合	新闻、论文、代码、专业 DB 的覆盖不同	提高召回率与领域覆盖
Domain filtering	研究常需要限制可信站点/官方域名	降低噪声与来源偏差
Search 与 Scrape 解耦	搜索 snippet 不等于证据全文	可独立替换抓取引擎
URL 去重/结果硬化	并行搜索容易重复或返回异常结构	减少成本与脏数据传播

从 2026 的 release notes 看，retriever hardening 已成为维护重点：v3.6.0 明确包含对 malformed results 的防御性修复；v3.4.2 加入 URL deduplication；v3.4.3 修复 Deep Research planning 使用所有配置 retrievers。这些修复揭示了生产系统的真实问题：Retriever 的失败通常不是“没有结果”，而是 schema 漂移、空字段、重复 URL、不同供应商返回形态不一致。

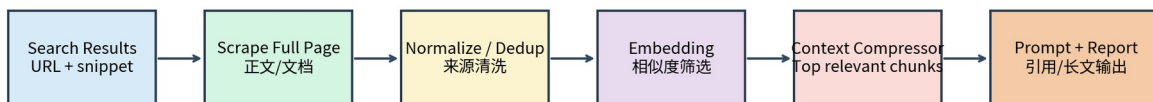
4. Scraper: 为什么必须读全文，而不是依赖 Search Snippet

GPT Researcher 的 pipeline 会在获取 URL 后进行 scraping，然后再做相关性压缩。Search snippet 适合候选发现，但通常缺上下文、引用关系、数字口径和限定条件；直接把 snippet 输入 LLM 会造成“看起来有证据，实际证据不完整”的伪 grounding。

- Search = candidate discovery；Scrape = evidence acquisition。两者职责不同。
- 网页正文、PDF、JS 渲染页面需要不同 scraper，项目因此维护独立 scraper 层。
- v3.6.0 的 scraper robustness 修复包含标题、PDF detection、临时文件、图像尺寸等边界问题，说明抓取层是可靠性高风险区。
- 本地文档模式同样遵循“解析原文 → 选相关内容 → 合成”的逻辑，而不是把整个文件无差别塞入上下文。

5. Context Manager: GPT Researcher 最值得拆的中间层

为什么它不是简单 Search → LLM：上下文工程层是核心



Source: context_manager.py and release notes on context compression/retriever hardening

图 3 从候选网页到可用研究上下文：Context Engineering Pipeline

当前 context_manager.py 明确将 ContextManager 定义为“处理 context retrieval、compression 和 similarity matching”。get_similar_content_by_query() 会创建 ContextCompressor，使用 researcher.memory.get_embeddings() 提供的 embedding，并传入 similarity_threshold，最终 async_get_context(query, max_results=10)。这说明最终送给写作模型的并不是所有抓取内容，而是一个 query-conditioned compressed context。

真正的系统瓶颈

Deep Research 的上限往往不由“搜到多少”决定，而由“能否把几十个网页压成一个不丢关键限定条件、又不超过 context budget 的证据集合”决定。Context Compression 既是成本优化，也是信息质量控制。

5.1 三类 Compressor

组件	输入	目标
ContextCompressor	抓取后的 pages	基于 query 选出最相关内容
VectorstoreCompressor	外部/持久化 vector store	从向量库检索相关上下文
WrittenContentCompressor	已写章节/草稿	防止多章节写作重复并复用已有内容

5.2 Context 管理的 know-how

- 先获取候选，再 query-conditioned 过滤；不要把 retrieval top-k 直接等同于 generation context。
- 对不同 context source 使用不同 compressor，而不是一个统一“万能 chunker”。
- 长报告写作时，需要把“已写内容”也纳入检索，以减少重复与跨章节矛盾。
- Deep Research 源码设有 MAX_CONTEXT_WORDS（当前代码中 25k words 安全边界），并优先保留较新的/相关上下文，体现显式 context budget。

6. Report Generation 与 Citation: Publisher 不只是“写作文”

官方 flow 文档显示，ReportGenerator.write_report() 最终调用 generate_report()，由 prompt family 根据 report_type 生成 prompt，再使用 smart LLM 基于 research context、report source、格式、tone、word count 等生成 Markdown 报告。WebSocket/服务器层随后可输出 PDF、DOCX、Markdown 等格式。

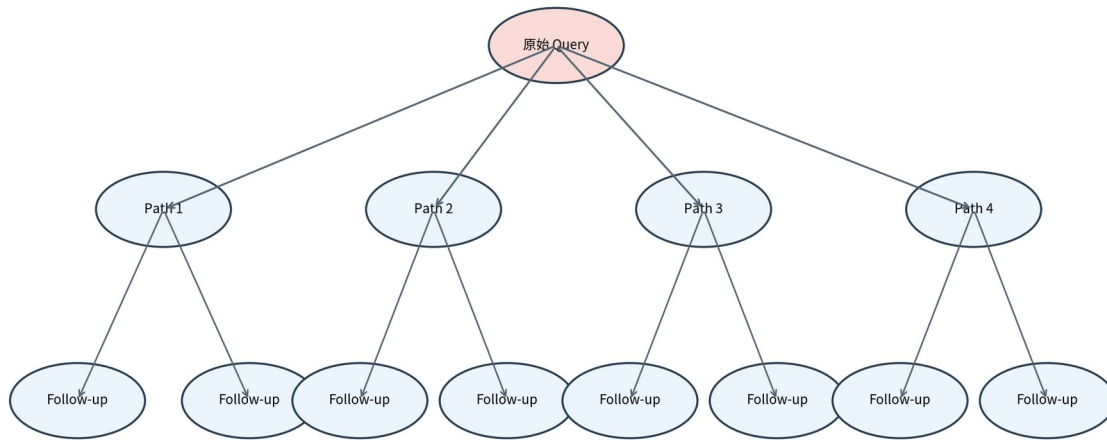
Citation 的关键在于 source tracking 从 research 阶段就存在。Deep Research 的源码也会从 research result 中解析 learnings、sourceUrl/citation，并将 citations 与 learning 绑定。换言之，引用不是写作末端“让模型自己补链接”，而是尽量随着 evidence 采集进入研究状态。

Know-how：引用必须前移

如果 citation 只在最后写作 prompt 才要求，模型很容易生成“有格式但不对应证据”的引用。更稳健的做法是 Evidence 对象从抓取阶段就携带 URL/metadata，压缩时保留 provenance，最后 writer 只消费可追踪 evidence。

7. Deep Research Mode: 从“并行子问题”升级到递归 Research Tree

Deep Research : Breadth × Depth 递归探索



每个节点：生成搜索查询 → 检索/抓取 → 提取 learnings + citations → 生成 follow-up → 继续递归
同时受 concurrency 与 MAX_CONTEXT_WORDS 等约束，避免成本与上下文无限增长。

Sources: gpt_researcher/skills/deep_research.py; official Deep Research docs

图 4 Deep Research 的递归 breadth × depth 结构

官方 Deep Research 文档将其描述为 tree-like exploration: Breadth 控制每层生成多少研究路径, Depth 控制每条路径继续追问多少层, Concurrency 控制同时进行的 research 操作。当前 docs 的默认值与代码/版本之间存在过调整的痕迹, 因此生产使用时应以实际 config 为准, 而不是硬编码文档数字。

7.1 DeepResearchSkill 的内部状态

- learnings: 从每一轮 research results 中抽取的关键发现。
- citations / research_sources: 将发现和来源绑定, 供最终合成。
- visited_urls: 避免在递归中重复访问同一 URL。
- context: 跨分支累积的研究上下文。
- ResearchProgress: 跟踪 current_depth、breadth、current_query、completed_queries 等, 用于实时进度。

7.2 递归循环的逻辑

1. 根据当前 query 生成多个 search queries, 每个 query 带 researchGoal。
2. 对每条 query 执行检索/研究, 并获得 context。
3. 让 strategic LLM 从 context 中抽取 learnings、citations 和 follow-up questions。
4. 将 follow-up questions 作为下一层 research query, 直到 depth 用尽或没有可继续问题。
5. 合并 learnings/citations/context, 并交给最终 report generation。

成本模型

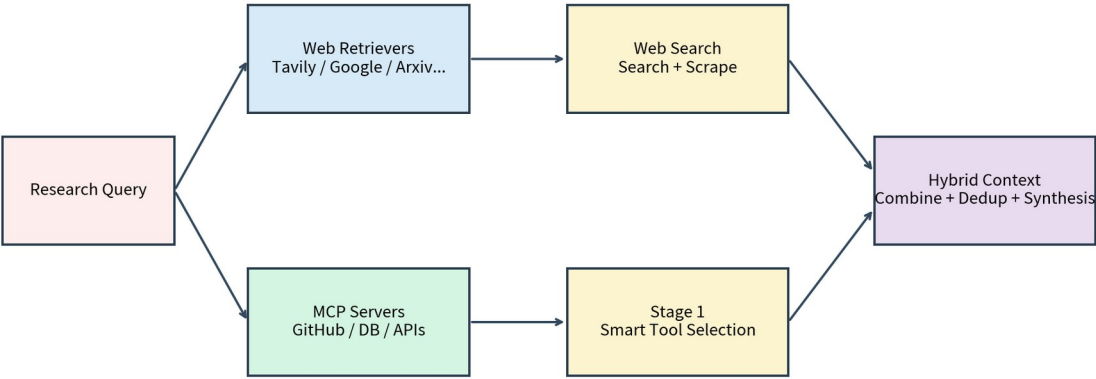
若不做剪枝，理论探索量接近 $O(B^D)$ 。实际系统依靠 concurrency、去重、失败跳过、上下文上限和 query 生成数量约束控制爆炸。Deep Research 的主要工程问题不是“怎么递归”，而是“什么时候停止、保留什么、如何防止重复”。

7.3 Deep Research 的容错设计

官方文档明确指出失败 query 会被跳过，单个 branch 失败不会停止整体研究。近期版本还修复了 JSON 输出解析 (json_repair + regex fallback) 和 oversized context 变空的问题。这里揭示一个重要经验：递归 agent 必须把“LLM 输出不是严格 JSON”视为常态，而不是异常。

8. MCP：把 Research Agent 从 Web 扩展到专业工具/数据源

MCP Hybrid Research：把“搜索引擎”扩展为“任意专业数据源”



Source: official MCP Integration docs

图 5 GPT Researcher 的 MCP Hybrid Retrieval 逻辑

GPT Researcher 的 MCP integration 采用官方文档所称的 two-stage intelligent approach：第一阶段由 LLM 根据 query 和可用 MCP servers 选择相关工具；第二阶段再用 query-specific arguments 执行工具。可以设置纯 MCP，也可以 RETRIEVER=tavily,mcp 等方式和传统 web search 组合。

MCP 策略	适合场景	Trade-off
Pure MCP	代码库、数据库、内部 API、专业数据源	覆盖受 MCP server 能力限制
Hybrid web + MCP	既需要公开信息，又需要内部/结构化事实	上下文更丰富，但去重与冲突处理更复杂
Fast strategy	重复子查询希望复用 MCP 结果	更省成本，但上下文针对性下降
Deep strategy	每个 sub-query 都需要专业工具重新查询	质量潜力高，成本/延迟更高

Know-why

MCP 的价值不是“又多一种工具协议”，而是把 GPT Researcher 的数据面从 Search Engine 抽象成 Tool/Data Source。这样 GitHub、数据库、财务 API、论文数据库、内部系统都可以进入同一 research → context → report pipeline。

9. Local Documents / Hybrid Research: 从 Web Research 到 Enterprise Research

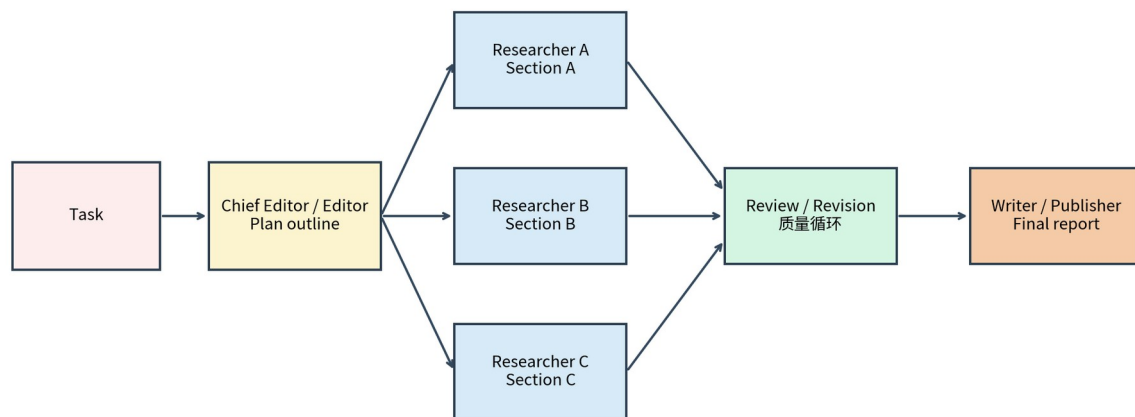
官方 Local Documents 文档支持 PDF、文本、CSV、Excel、Markdown、PowerPoint、Word。

report_source 可以设为 local；项目也支持 local + web hybrid。其工程意义是把相同的“子问题 → 相关内容 → evidence → report”逻辑复用到私有知识库，而不是另写一套完全不同的 RAG。

- Web source 强在时效性与广度；Local source 强在组织私有事实与上下文。
- Hybrid 是企业研究最实际的形态：内部材料回答“我们是谁/现状”，Web 回答“外部市场/竞争/最新变化”。
- 项目在 Deep Agents benchmark 中报告 private-docs coverage 提升，但这是项目方自测，应作为实现线索，不应视为独立第三方结论。

10. Multi-Agent: 为什么还需要 Editor / Reviewer / Revisor

Multi-Agent：把“研究”从单任务循环提升为“章节级并行 + 审核修订”



Sources: official LangGraph docs; deep_agents/README.md

图 6 章节级 Multi-Agent 研究与审核流程

官方 LangGraph/AG2 文档将复杂长报告拆为多个角色：Editor 规划 outline；多个 Researcher 并行完成章节；Reviewer 根据 criteria 审核；Revisor 按反馈修订；Writer 汇总引言、结论、references；Publisher 输出 PDF/DOCX/Markdown。

10.1 这不是“角色越多越智能”

Multi-Agent 的真正收益来自三个结构性变化：第一，章节级 context isolation，避免所有原始研究塞入一个 context；第二，research tasks 并行化；第三，把质量控制显式建模为 review/revision loop。角色名称只是组织方式。

10.2 代价

- 更多 LLM calls → 成本与延迟显著增加。
- Review/revision 如果没有 max revision，会发生循环；2026 release 专门增加上限和 sentinel hardening。
- 章节独立研究会产生来源重复、术语不一致和跨章节冲突，需要最终 writer 做全局统一。
- “Reviewer 是另一个 LLM” 并不自动等价于事实验证；它更像结构化自审。

11. v3.6.0 Deep Agents Integration：研究引擎正在被“工具化”

v3.6.0 新增 deep_agents 示例，项目将 GPT Researcher 作为 research engine 嵌入 LangChain Deep Agents harness。与早期 LangGraph hardcoded graph 相比，planning、delegation、context management 更多由 harness 内置机制承担：write_todos 做计划，task 工具生成 researcher subagents，草稿写入 filesystem 做 context offload，Chief Editor 再读取/审阅/合并。

能力	早期 LangGraph Multi-Agent	Deep Agents 示例
Planning	EditorAgent node + ResearchState	write_todos 工具
并行研究	每节一个嵌套 subgraph	task 工具 spawn researcher subagents
Context 管理	state keys 在节点间传递	filesystem offload + subagent isolation
Review/Revision	Reviewer/Revisor 显式节点循环	Chief Editor 读草稿并再次委派
Publish	PublisherAgent	report.md 在文件系统组装

趋势判断（源码推断）

GPT Researcher 正从“完整 Research App”同时演化为“可被更大 Agent Harness 调用的 research capability”。这与 DeerFlow 2.0 将 Deep Research 变为 Skill 的方向相似：长任务 Agent 的核心会逐渐变成通用 harness + 专业 research tool/skill，而不是每种任务都写一套固定图。

12. Benchmark 与评测：应该看什么，不应该只看什么

Deep Research Agent 的评测不能只看最终答案准确率，因为系统输出是长报告。DeepResearch Bench 将评估拆为 report quality（RACE）与 information retrieval / citation（FACT）等维度；2026 年 DeepResearch Bench II 又进一步使用大量细粒度 rubrics 评估信息召回、分析与呈现。

12.1 GPT Researcher v3.6 Deep Agents 自报结果

Metric	Raw search tool	GPT Researcher tools	解释
RACE overall	0.503	0.512	报告质量小幅提高
Verified citations/report	18.6	35.2	项目方报告约 +89%
Citation precision	53.9%	56.0%	精确率小幅提高
Zero verifiable citation reports	2/10	0/10	证据覆盖改善

这些数据来自 deep_agents/README.md 对 DeepResearch Bench 的项目方实验：相同 harness、model、prompt、step budget，仅对比 raw Tavily search 与 GPT Researcher research tools。它很好地说明“读全文 + 预先综合/引用”可能比只返回 search snippets 提供更多可验证证据，但样本规模为 10 tasks，且实验由项目方维护，因此应标注为 self-reported benchmark。

12.2 更合理的评测矩阵

维度	建议指标	为什么
Retrieval	Recall / source diversity / freshness	是否找到该找的证据
Citation	precision / coverage / effective citations	链接是否真的支持 claim
Synthesis	contradiction handling / completeness	能否正确整合冲突信息
Writing	structure / redundancy / coherence	长报告可读性
Agent behavior	search depth / tool efficiency / loops	过程是否高效可控
Cost	token / API / latency	生产可行性
Robustness	empty result / malformed HTML / tool failure	真实网络环境稳定性

13. Reliability / Security / Production Hardening

v3.6.0 的 change log 对理解真实 Agent 工程尤其有价值：sanitize untrusted content、retriever malformed-result guards、scraper robustness、cost tracking、pipeline robustness、multi-agent bounded revision loops、MCP/config fixes。它们说明 Agent 的可靠性不是单纯“Prompt 写得更严谨”，而是需要在每个 IO boundary 做程序级防御。

Failure mode	需要的机制	GPT Researcher 当前线索
Retriever 返回异常 schema	normalize + validate + skip bad item	v3.6 retriever hardening
网页恶意/不可信文本	sanitize / treat as data, not instruction	v3.6 security changes
Scraper 超时/PDF/临时文件	timeout + fallback + cleanup	v3.4.3/v3.6 fixes
LLM 非法 JSON	json_repair + regex fallback	v3.5 deep_research fix

上下文过长	compression + truncation + explicit budget	context manager + deep context limit
Review loop 不停止	MAX_REVISIONS / exact sentinel	v3.5.1/v3.6
没有有效内容却生成报告	abstain / fail gracefully	v3.5.1
成本不可见	per-step usage/cost tracking	v3.4.3/v3.6

Know-how：把 LLM 当不可靠组件

LLM 输出、搜索 API、网页 HTML、MCP tool、PDF parser 都应该被视为“不可靠外部输入”。Production Agent 的主工作不是让 happy path 更漂亮，而是让每一个失败点都能被检测、降级、跳过、重试或停止。

14. GPT Researcher 与传统 RAG / STORM / DeerFlow 的结构对比

系统	核心对象	搜索策略	Context 管理	长任务编排	最值得学
传统 RAG	Question → Documents	通常单轮 top-k	chunk + retrieval	弱	外部知识 grounding
GPT Researcher classic	Research task	多 sub-query + parallel	scrape + compressor	中	研究流水线/证据聚合
GPT Researcher deep	Research tree	breadth × depth 递归	跨分支 learnings/context	中-强	adaptive follow-up search
STORM	Long-form article	多视角 question asking	knowledge curation + outline	中	pre-writing / outline
GPT Researcher multi-agent	Report sections	章节级并行 research	section isolation + review	强	review/revision loop
DeerFlow 2.0	Long-horizon task	Research 是 Skill	filesystem/ sandbox/memory	强	通用 SuperAgent harness

核心差异可以概括为：RAG 关注“检索增强一次生成”，GPT Researcher 关注“研究流程”；STORM 更强调 pre-writing 的多视角与 outline；DeerFlow 2.0 则把 research 从主流程降为通用 Agent 的一个 Skill。GPT Researcher v3.6 的 Deep Agents 示例正在向后者靠近。

15. 设计中最可复用的 Know-how

15.1 研究问题必须可执行

Research plan 不是文章目录。目录标题往往不能直接搜索，例如“行业影响”太宽；可执行 research query 应包含对象、时间、比较维度、所需证据类型。Planner 的目标是把语义任务转成可检索动作。

15.2 Search 与 Read 必须分层

Search engine 是候选发现器，Scraper/Document loader 才是 evidence reader。只消费 snippet 的 Agent 往往 citations 多但证据浅；全文读取虽然更贵，但能保留限定条件与上下文。

15.3 Context Compression 是 Research Agent 的“工作记忆管理”

长任务的核心不是拥有无限 context，而是持续决定哪些信息值得进入下一轮模型调用。GPT Researcher 使用 embedding-based compression、vectorstore compressor 和 written-content compressor，体现“不同阶段使用不同记忆压缩器”的思路。

15.4 Evidence 必须带 provenance

URL、标题、source metadata 应和 evidence 一起流动。不要在最后一刻根据报告文本重新搜索 citation；那会造成 claim-source mismatch。

15.5 并行只适合相对独立的研究单元

Sub-query/section research 可以并行，但最终 synthesis、冲突处理和 narrative organization 需要一个全局视角。过度 parallelization 会增加重复搜索和不一致。

15.6 递归搜索必须有 stopping policy

Depth 只是最粗的停止条件。更成熟的 stopping policy 应考虑：新增信息增益、未覆盖问题数、来源重复率、成本预算、剩余时间和 confidence。

15.7 把“研究”做成可调用 Tool，比把整个 App 写死更有长期价值

Deep Agents 示例将 GPT Researcher 暴露为 quick_search 和 deep_research 工具，让更上层 harness 负责 plan/delegate/filesystem。这种架构更适合未来 Agent Runtime：专业能力模块化，上层负责生命周期与上下文。

16. 如果从零实现一个 GPT Researcher-like MVP

6. 定义 ResearchState: query、sub_queries、evidence、visited_urls、cost、status。
7. 实现 query_planner(query) → 3-5 个差异化、可搜索 sub-query。
8. 实现 Retriever interface: search(query) → [{url,title,snippet,metadata}]。
9. 实现 Scraper interface: fetch(url) → normalized text，并做 timeout / PDF / JS fallback。
10. 对候选文档按 sub-query 做 chunk + embedding similarity，控制 max relevant chunks。
11. Evidence schema 必须保存 source URL/title/retrieved_at，不允许 context 丢 provenance。
12. 并行执行 sub-queries，做 URL dedupe 与 per-domain limit。
13. 先生成 research context，再独立调用 writer；不要把 search 与 writing 混成一个黑箱 prompt。
14. 加入 citation checker：检查每个引用 URL 是否存在、是否在 evidence 集合、是否能支持对应 claim。
15. 最后再加 recursive deep mode: follow-up question + depth/breadth + stopping rule。

MVP 原则

先做“单 Researcher + 强 Context/Evidence pipeline”，再做 Multi-Agent。多数系统的质量瓶颈在 retrieval/read/compression，而不是缺少第 7 个 Agent 角色。

17. Production 版本建议：在 GPT Researcher 基础上继续改善什么

方向	建议	Know-why
Source quality	引入 source prior / domain trust / primary-source preference	多源数量不能代替来源质量
Claim-evidence graph	报告生成后构建 claim→evidence 映射并二次验证	提高 citation precision
Adaptive stopping	用信息增益和 coverage，而非固定 depth	降低递归成本
Conflict detection	对同一事实的数字/日期/定义做冲突聚类	多源检索会带来矛盾
Search policy eval	记录每步 query、点击、抓取、保留/丢弃原因	才能优化 Agent policy
Context observability	显示 token/word budget、每来源占比、压缩损失	Context 是隐形瓶颈
Caching	按 URL+etag/时间缓存正文，按 query 缓存搜索	降低重复 research 成本
Sandbox/security	网页内容视为数据，工具权限最小化	防 prompt injection / tool misuse
Deterministic artifacts	evidence.json + report.md + trace.json	便于复现与审计

18. 源码阅读地图：按“最短路径”理解项目

顺序	文件/目录	看什么
1	README.md	项目定位、Architecture、Feature 与运行入口
2	.claude/references/flows.md	当前维护者总结的 end-to-end code/data flow
3	gpt_researcher/agent.py	GPTResearcher 根对象与 conduct_research/write_report
4	gpt_researcher/skills/researcher.py	sub-query 研究、MCP/web context 合并
5	gpt_researcher/skills/context_manager.py	compression / embeddings / vectorstore
6	gpt_researcher/context/	ContextCompressor 的具体策略
7	gpt_researcher/retrievers/	Retriever interface 与各搜索后端

8	gpt_researcher/scrapper/	全文抓取与页面/PDF 处理
9	gpt_researcher/actions/ report_generation.py	Prompt → final report
10	gpt_researcher/skills/deep_research.py	breadth/depth 递归与 context budget
11	multi_agents/	LangGraph/AG2 multi-agent 角色与 revision loop
12	deep_agents/README.md + agent.py	2026 harness 化方向与 benchmark

19. 研究结论

GPT Researcher 最重要的贡献不是提出全新的单一算法，而是把多个当时分散的思想组合成一个可运行的 Research Agent：Plan-and-Solve 提供任务分解，RAG 提供外部知识 grounding，异步编排提供并发，scraping 让模型读取全文，context compression 控制工作记忆，source tracking 提供引用，后续又通过 recursive search、MCP 和 Multi-Agent 扩展研究深度与数据边界。

从 2023 到 2026 的演化也说明一个更普遍的 Agent 工程规律：第一阶段追求“让模型会用工具”；第二阶段追求“让多个工具形成 workflow”；第三阶段追求“把 workflow 的中间数据压缩成高价值 context”；第四阶段追求“将专业 workflow 变成可被通用 Agent Harness 调用的 Skill/Tool”；最终竞争点变成可靠性、证据质量、成本与可观测性。

最终 Know-why

Deep Research 的本质不是“多搜几次”，而是维护一个动态 Research State：根据当前证据、未覆盖问题和不确定性，持续决定下一步最有信息价值的检索动作，并把新证据压缩回有限工作记忆，直到继续搜索的边际收益低于成本。

附录 A：关键配置与调优思路

参数/能力	作用	调优建议
RETRIEVER	搜索/检索来源	按任务选择，不要盲目堆多 retriever
DEEP_RESEARCH_BREADTH	每层并行研究路径	提高 coverage，同时提高搜索成本
DEEP_RESEARCH_DEPTH	递归层数	适合需要追踪因果/演化/细节的任务
DEEP_RESEARCH_CONCURRENCY	并发	受 API rate limit 与主机资源约束
TOTAL_WORDS	报告长度	应和 evidence 量匹配，避免“为了长而长”
SIMILARITY_THRESHOLD	相关性筛选门槛	太低会污染 context，太高会漏证据
DOC_PATH	本地资料目录	企业/私有 research 的基础

MCP_STRATEGY	MCP 使用策略	fast 重复利用, deep 每 sub-query 重新查询
LANGCHAIN_TRACING_V2	观测	生产调优强烈建议开启等价 tracing

附录 B：来源与参考资料

ID	来源	URL	用途
S1	GPT Researcher GitHub repository	https://github.com/assafelovic/gpt-researcher	项目定位、README、当前源码
S2	GitHub repository API metadata (2026-08-13)	https://api.github.com/repos/assafelovic/gpt-researcher	Stars/Forks/更新时间/License 等当前状态
S3	GPT Researcher Releases - v3.6.0	https://github.com/assafelovic/gpt-researcher/releases	2026 可靠性、安全与 Deep Agents 演化
S4	Research Flow & Data Flow	https://github.com/assafelovic/gpt-researcher/blob/main/.claude/references/flows.md	当前 end-to-end code/data flow
S5	ContextManager source	https://github.com/assafelovic/gpt-researcher/blob/main/gpt_researcher/skills/context_manager.py	Context retrieval/compression/similarity
S6	DeepResearchSkill source	https://github.com/assafelovic/gpt-researcher/blob/main/gpt_researcher/skills/deep_research.py	递归搜索、learnings/citations、context budget
S7	Official Deep Research docs	https://docs.gptr.dev/docs/gpt-researcher/gptr/deep_research	breadth/depth/concurrency 与使用方式
S8	Official Search Engines docs	https://docs.gptr.dev/docs/gpt-researcher/search-engines	Retriever 组合与搜索后端
S9	Official MCP Integration docs	https://docs.gptr.dev/docs/gpt-researcher/retrievers/mcp-configs	two-stage MCP、hybrid strategy
S10	Official Local Documents docs	https://docs.gptr.dev/docs/gpt-researcher/context/local-docs	本地/混合研究
S11	Official LangGraph Multi-Agent docs	https://docs.gptr.dev/docs/gpt-researcher/multi_agents/langgraph	Editor/Researcher/Reviewer/Revisor/Writer/Publisher
S12	Deep Agents x GPT Researcher	https://github.com/assafelovic/gpt-researcher/blob/main/deep_agents/README.md	2026 harness 集成与项目方 benchmark
S13	Plan-and-Solve Prompting, ACL 2023	https://aclanthology.org/2023.acl-long.147/	任务先规划再执行的基础思路
S14	Retrieval-Augmented Generation, NeurIPS 2020	https://papers.nips.cc/paper_files/paper/2020/hash/6b493230205f780e1bc26945df7481e5-Abstract.html	外部非参数知识 + 生成
S15	STORM, NAACL 2024	https://aclanthology.org/	多视角研究、pre-writing 与长文组

		2024.naacl-long.347/	织
S16	DeepResearch Bench (2025 / ICLR 2026)	https://arxiv.org/abs/2506.11763	长报告 Research Agent 评测：质量与引用
S17	DeepResearch Bench II (2026)	https://arxiv.org/abs/2601.08536	细粒度 rubric: recall / analysis / presentation

说明：本报告未将项目 README 中“更多来源即可显著降低错误”的表述视为已被严格证明的统计结论。多源可以降低单源偏差风险，但来源可能存在复制、共同错误、SEO 内容污染或同源传播，因此生产系统仍需来源质量建模、冲突检测与 claim-level evidence verification。

附录 C：证据等级与阅读提示

标记	定义	本报告用法
官方事实	README/docs/release/源码直接支持	当前功能、类/函数、配置、release
源码推断	从调用关系/数据结构推导	设计意图、模块边界、趋势
工程评价	结合 Agent/RAG 工程经验的判断	优缺点、改进建议、know-how
项目方 benchmark	项目维护者自行设计/运行的实验	作为实现线索，明确标注非独立第三方